Projet Docker

Barème : 20 points

Temps de réalisation : entre 4 heures et 6 heures

Critères d'évaluation :

- Compréhension et suivi des instructions.
- Exactitude des commandes Docker.
- Structure et organisation du Dockerfile.
- Fonctionnement de l'image Docker créée.
- Respect des bonnes pratiques Docker.

Objectif : créer un Dockerfile pour un projet MEAN (MongoDB, Express, Angular, Node).

Vous apprendrez à configurer un environnement Docker complet pour une application web *full stack*.

Instructions

Vous allez compléter un Dockerfile, fourni en annexe, en suivant une série d'instructions numérotées et détaillées, pour construire et configurer une application MEAN.

Chaque instruction correspond à une étape clé dans la construction de votre image Docker. Suivez-les attentivement et complétez les parties manquantes.

Barème de points par instruction :

1. Utilisation de l'image Node. 2 points

– Sélection correcte de l'image Node version 19 « bullseye ».

2. Copie des fichiers package.json et package-lock.json. 2 points

– Copie correcte des fichiers dans le conteneur.

3. Installation des dépendances. 3 points

– Installation correcte avec prise en compte des dépendances *legacy*.

4. Définition du répertoire de travail. 2 points

– Configuration correcte du répertoire de travail pour /app.

5. Copie du code front-end. 2 points

– Copie adéquate de tout le code front-end dans le conteneur.

6. Construction de l'application Angular. 3 points

– Construction réussie en mode production.

7. Utilisation d'une image Node plus légère. 1 point

– Choix approprié de l'image Node version 16 avec Alpine.

8. Copie du code back-end. 1 point

– Copie adéquate du code back-end dans le conteneur.

9. Installation des dépendances du back-end. 1 point

– Installation correcte des dépendances du back-end.

10. Copie des fichiers Angular compilés. 1 point

– Copie correcte des fichiers depuis l'étape « builder ».

11. Exposition du port. 1 point

– Exposition correcte du port utilisé par l'application Express.js.

12. Définition de la commande de démarrage. 1 point

– Configuration correcte de la commande pour démarrer l'application.



Ressources
Vous êtes encouragé·e à effectuer des recherches indépendantes pour réaliser cet exercice.

L'utilisation de ChatGPT pour des solutions directes est interdite.

Conseils

- Choisissez un endroit calme pour travailler et favoriser votre concentration.
- Assurez-vous de bien comprendre chaque instruction avant de commencer.
- Testez régulièrement votre Dockerfile pour vérifier son fonctionnement.
- Effectuez des recherches indépendantes pour approfondir votre compréhension.
- Avant de soumettre votre travail, relisez-vous pour vous assurer de la clarté et de l'absence d'erreurs.

À propos (MongoDB – Express – Angular – Node)

Le MEAN stack, composé de MongoDB, Express.js, Angular.js et Node.js, forme la base d'une application web *full stack*. Ce projet sert de point de départ idéal pour en créer une. Le projet démontre également une approche réalisable pour faire fonctionner une application en direct sur Docker dans des environnements de développement et de production.

Voici l'architecture de l'application lorsqu'elle est en cours d'exécution.

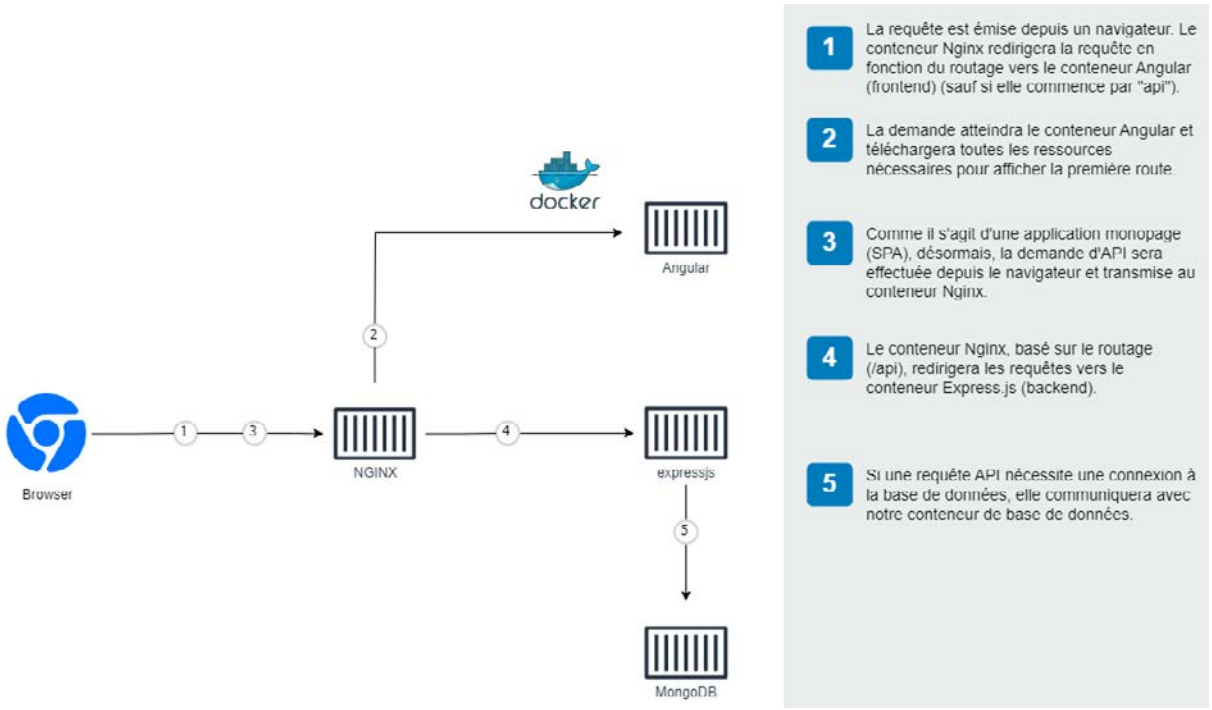


Fig. 1 Architecture de l'application en cours d'exécution © Skill and You

Pour un démarrage rapide

Clonez le dépôt, naviguez jusqu'au dossier racine et exécutez :

```
docker-compose -f 'docker-compose.nginx.yml' up
git clone https://github.com/Skill-You/docker_mean.git
cd docker_mean
docker-compose -f 'docker-compose.yml' up
```

Dossiers du projet

Les applications se trouvent dans les *frameworks*/librairies JavaScript suivants :

Tableau n°1 Dossiers du projet

DOSSIER	DESCRIPTION
frontend	App front-end utilisant Angular.js https://github.com/Skill-You/docker_mean/tree/main/frontend
api	API utilisant Express.js https://github.com/Skill-You/docker_mean/tree/main/backend
loadbalancer	*Load balancer* utilisant NGINX https://github.com/Skill-You/docker_mean/tree/main/loadbalancer
mongo	Configuration de l'image MongoDB https://github.com/Skill-You/docker_mean/tree/main/mongo

À PROPOS DU PROJET

L'application web présentée ici est simple, mais fonctionnelle. Elle comprend une page d'inscription et une page de connexion entièrement opérationnelles, ainsi qu'une démonstration complète des fonctionnalités CRUD (créer, lire, mettre à jour, supprimer) qui incorpore le routage Angular et des exemples d'utilisation de l'API REST avec Express.js. De plus, les services REST sont sécurisés en implémentant des JWT (jetons web JSON) pour une sécurité renforcée.

– **Angular (16.0.1)**

La partie front-end de ce projet est construite avec Angular, qui représente le « A » dans MEAN stack. Pour activer le rendu côté serveur (SSR), nous avons choisi d'utiliser l'image Alpine de Node.js au lieu d'une image Docker légère comme NGINX pour exécuter l'application Angular.

Le projet inclut des exemples de code pour diverses fonctionnalités telles que : inscription d'utilisateur, connexion, gestion de profil, exemple complet de CRUD.

De plus, il y a des exemples de code pour implémenter une garde d'authentification, des services, des intercepteurs HTTP, des résolveurs et une authentification JWT.

– **Express (4.17.1)**

Dans le MEAN stack, « E » représente Express. Tous les services REST sont développés en utilisant Express.js.

Il contient des exemples pour : connexion MongoDB et validation de schéma avec Mongoose, mise en œuvre JWT pour l'autorisation, routage de l'API, API d'inscription et de connexion d'utilisateur, exemple complet de CRUD.

– **MongoDB**

Nous utilisons MongoDB pour la base de données. MongoDB est un programme de bases de données orientées documents et multiplateformes. Classifié comme un programme de bases de données NoSQL, MongoDB utilise des documents de type JSON avec des schémas optionnels.

Script de données initiales :

https://github.com/Skill-You/docker_mean/blob/main/mongo/init-db.d/01.Seed.sh

Script de création d'utilisateur de base de données :

https://github.com/Skill-You/docker_mean/blob/main/mongo/init-db.d/02.Users.sh

– **NGINX**

Remarque : seulement si vous utilisez Docker.

Nous avons utilisé NGINX comme équilibreur de charge au cas où le front-end et l'API doivent être exposés sur le même port. Pour la configuration, veuillez vérifier nginx.conf (https://github.com/Skill-You/docker_mean/blob/main/loadbalancer/nginx.conf).

Pour commencer le projet avec ou sans Docker

Avec Docker

– Prérequis :

Installez la dernière version de Docker Desktop : https://www.docker.com/products/docker-desktop/

– Mode de production :

Pour le mode de production, il y a deux options :

1. Utilisation de deux conteneurs (Express (front-end et API) et Mongo)

```
git clone https://github.com/Skill-You/docker_mean.git
cd docker_mean
docker-compose -f 'docker-compose.yml' up
```

2. Exécutez simplement ci-dessous, car Docker considère le nom de fichier par défaut « docker-compose.yml ».

```
docker-compose up
```

Cela exécutera le front-end et l'API sur http://localhost:3000. Vous pouvez également accéder à MongoDB sur le port 27017.

À propos du fichier Docker Compose

L'objectif principal de ce projet est de montrer la manière possible de faire fonctionner une application réelle (MEAN stack) avec Docker.

Le fichier docker-compose.yml utilise deux conteneurs :
– express : pour héberger le front-end (Angular) et le back-end API (Express.js) ensemble ;
– base de données : pour héberger MongoDB.



Utilisation de MongoDB comme service géré
Si, dans le cas ci-dessus, nous utilisons MongoDB comme service géré, nous n'aurons besoin que d'un seul conteneur.

```
version : « 3.8 » # spécifiez la version de docker-compose
# Définissez les services/conteneurs à exécuter
services :
express : #nom du deuxième service
build : # spécifiez le répertoire du Dockerfile
context : .
dockerfile : dockerfile
container_name : mean_angular_express
ports :
– « 3000:3000 » #spécifiez les ports de transfert
# Ci-dessous, les variables d'environnement de la base de données pour l'API sont utiles lorsque vous devez utiliser la base de données comme service géré
environment :
– SECRET = Thisismysecret
– MONGO_DB_USERNAME = admin-user
– MONGO_DB_PASSWORD = admin-password
– MONGO_DB_HOST = database
– MONGO_DB_PORT =
– MONGO_DB_PARAMETERS = ? authSource = admin
– MONGO_DB_DATABASE = mean-football
links :
– database
database : # nom du troisième service
image : mongo : latest # spécifiez l'image pour construire le conteneur
container_name : mean_mongo
environment :
– MONGO_INITDB_ROOT_USERNAME = admin-user
– MONGO_INITDB_ROOT_PASSWORD = admin-password
– MONGO_DB_USERNAME = admin-user1
– MONGO_DB_PASSWORD = admin-password1
– MONGO_DB = mean-football
volumes :
–./mongo :/home/mongodb
–./mongo/init-db.d/:/docker-entrypoint-initdb.d/
–./mongo/db :/data/db
ports :
– « 27017 : 27017 » # spécifiez les ports de transfert
```

Sans Docker

– Prérequis :

1. Installez la dernière version de Node.js : https://nodejs.org/en/.

2. Installez Nodemon comme paquet global (pour exécuter Express.js en mode de développement) : npm install -g nodemon.

3. Installez Angular CLI : npm install -g @angular/cli (facultatif).

4. Installez MongoDB localement ou inscrivez-vous pour un compte géré gratuit : https://www.mongodb.com/fr-fr/cloud/atlas/register

5. Avant d'exécuter le projet, assurez-vous que vous pouvez vous connecter à MongoDb. Vous pouvez utiliser Robo 3T pour cela : https://robomongo.org/

– Exécution du projet :

Clonez le projet et exécutez npm install dans les dossiers frontend et api.

```
git clone https://github.com/Skill-You/docker_mean.git
cd docker_mean/frontend
npm i
npm start
cd docker_mean/backend
npm i
npm start
```

Pour passer les variables d'environnement (détails de la base de données) dans l'API, naviguez jusqu'au dossier api, renommez « .env.example » en « .env » et actualisez-y vos détails MongoDB.

Vous pouvez également exécuter npm run dev-server depuis le dossier frontend pour exécuter le front-end et l'API ensemble.

Cela exécutera l'API sur http://localhost:3000 et le front-end sur http://localhost:4200.



Feuille de route
Consultez les problèmes ouverts pour une liste des fonctionnalités proposées (et des problèmes connus) : https://github.com/Skill-You/docker_mean/issues?q=is%3Aissue+is%3Aopen+sort%3Aupdated-desc

Contributions

Les contributions font de la communauté open source un endroit incroyable pour apprendre, s'inspirer et créer. Toutes les contributions que vous faites sont grandement appréciées.
– Forkez le projet.
– Créez votre branche de fonctionnalités (git checkout -b feature/AmazingFeature).
– Commitez vos changements (git commit -m 'Add some AmazingFeature').
– Poussez vers la branche (git push origin feature/AmazingFeature).
– Ouvrez une demande de tirage (*pull request*).



Licence
MIT
https://github.com/Skill-You/docker_mean/blob/main/LICENSE/